

MODUL PRAKTIKUM PEMROGRAMAN BACKEND LANJUT

Pertemuan 6 — Authorization & Role Based Access Control

Program Studi	D4 Teknik Informatika — Fakultas Vokasi, Universitas Airlangga
Mata Kuliah	Praktikum Pemrograman Backend Lanjut (2 SKS) — Semester 5
Alokasi Waktu	100 menit praktikum terbimbing + 100 menit tugas mandiri

Tujuan Pembelajaran

Setelah menyelesaikan pertemuan ini, mahasiswa diharapkan mampu:

1. Membedakan authentication dan authorization, serta menjelaskan mengapa keduanya tidak boleh dicampur dalam satu pemeriksaan.
2. Merancang skema Role Based Access Control (RBAC) dalam bentuk tabel relasional: roles, permissions, dan role_permissions.
3. Mengimplementasikan pemeriksaan permission sebagai middleware untuk keputusan akses yang tidak bergantung pada isi data.
4. Mengimplementasikan pemeriksaan kepemilikan (ownership) di layer service untuk keputusan akses yang bergantung pada isi data.
5. Menerapkan prinsip fail closed dan menjelaskan akibatnya bila prinsip tersebut dilanggar.
6. Menggunakan status code 401 dan 403 pada situasi yang tepat.

Koreksi jujur terhadap Modul 5

Pada Modul 5 kita sudah menyisipkan field role ke dalam JWT. Namun sampai akhir modul tersebut, tidak ada satu pun endpoint yang benar-benar membaca field itu untuk mengambil keputusan.

Artinya, sistem yang kita bangun sampai sekarang memiliki lubang besar: siapa pun yang berhasil login — termasuk mahasiswa yang baru saja mendaftar sendiri lewat `/auth/register` — dapat melihat daftar seluruh user, mengubah data orang lain, bahkan menghapusnya. Autentikasi yang kuat tanpa authorization hanya memastikan bahwa penyusup tersebut punya nama.

Pertemuan ini menutup lubang tersebut.

A. Landasan Teori

A.1 Authentication dan Authorization

Dua istilah ini sering tertukar karena sama-sama berhubungan dengan keamanan dan sama-sama berawalan "auth". Pembedanya sebenarnya sederhana dan dapat dihafal dengan dua pertanyaan:

	Authentication	Authorization
Pertanyaan	Siapa Anda?	Anda boleh melakukan apa?
Dijawab dengan	Username dan password, lalu ditukar menjadi token	Role dan permission yang melekat pada identitas tadi

	Authentication	Authorization
Gagal berarti	401 Unauthorized	403 Forbidden
Dibahas pada	Modul 5	Modul 6 (pertemuan ini)
Urutan	Selalu didahulukan	Hanya bermakna setelah identitas diketahui

Urutannya tidak dapat dibalik. Tidak mungkin memutuskan "boleh atau tidak" sebelum tahu "siapa". Karena itu dalam kode kita, `RequireAuth` selalu dipasang sebelum `RequirePermission`. Bila urutannya tertukar, pemeriksaan permission akan membaca identitas yang belum ada.

A.2 Tiga Model Kontrol Akses

Ada beberapa cara menjawab pertanyaan "boleh melakukan apa". Tiga yang paling umum:

Model	Cara kerja	Cocok untuk	Kelemahan
ACL (Access Control List)	Setiap objek menyimpan daftar siapa saja yang boleh mengaksesnya	Sistem berkas, sharing dokumen	Jumlah entri membengkak seiring jumlah objek
RBAC (Role Based Access Control)	User diberi role; role memiliki sekumpulan permission	Aplikasi bisnis pada umumnya	Kaku bila hak akses bergantung pada konteks data
ABAC (Attribute Based Access Control)	Keputusan diambil dari atribut user, objek, dan lingkungan (waktu, lokasi, status)	Kebijakan yang rumit dan berubah-ubah	Sulit dibaca, sulit diuji, sulit dijelaskan ke auditor

Kita memakai RBAC karena berada di titik seimbang: cukup fleksibel untuk aplikasi nyata, tetapi masih cukup sederhana untuk dibaca satu layar. Perlu dicatat, RBAC murni tidak dapat menjawab pertanyaan seperti "bolehkah user ini mengubah datanya sendiri" — pertanyaan itu bergantung pada isi data, bukan pada role. Untuk kasus tersebut kita menambahkan pemeriksaan kepemilikan, yang sebenarnya adalah sepotong kecil ABAC. Kombinasi inilah yang dipakai hampir semua aplikasi produksi.

A.3 Anatomi RBAC

RBAC menyisipkan satu lapisan perantara antara user dan hak akses:



Mengapa tidak langsung user → permission saja? Bayangkan ada 400 mahasiswa dan suatu hari kebijakan berubah: asisten praktikum kini boleh menghapus data. Tanpa lapisan role, Anda harus menyunting 400

baris. Dengan role, Anda menambahkan satu baris pada tabel `role_permissions`. Lapisan perantara itu tampak seperti kerja tambahan di awal, tetapi ia adalah satu-satunya alasan sistem masih dapat dirawat setahun kemudian.

A.4 Penamaan Permission

Nama permission sebaiknya mengikuti pola yang konsisten. Kita memakai pola **domain:action:scope**:

Permission	Arti
user:list	Melihat daftar seluruh user
user:read:any	Melihat data user mana pun, bukan hanya dirinya sendiri
user:update:any	Mengubah data user mana pun
user:delete	Menghapus user
role:assign	Mengubah role milik user lain

Perhatikan sufiks `:any`. Sufiks itu menegaskan bahwa permission tersebut berbicara tentang *data orang lain*. Hak atas data sendiri tidak perlu permission — ia mengalir dari kepemilikan. Inilah sebabnya role user pada tabel di atas tidak memiliki permission satu pun, tetapi tetap dapat membaca dan mengubah profilnya sendiri.

A.5 Fail Closed

Ketika sistem tidak yakin — role tidak dikenal, nama permission salah ketik, data pemetaan gagal dimuat — ada dua kemungkinan sikap:

Sikap	Perilaku saat ragu	Akibat bila salah
Fail open	Izinkan	Salah ketik satu huruf membuka akses ke seluruh dunia; kesalahan tidak terlihat karena semuanya "berfungsi"
Fail closed	Tolak	Salah ketik satu huruf menutup akses; kesalahan langsung terlihat dan langsung dilaporkan pengguna

Kita memilih fail closed. Konsekuensinya akan terasa di kode: fungsi `Can()` mengembalikan `false` untuk role yang tidak dikenal, permission yang tidak dikenal, bahkan untuk penerima `nil`. Tidak ada satu pun jalur di dalamnya yang mengembalikan `true` karena "tidak menemukan alasan untuk menolak".

A.6 Di Mana Pemeriksaan Diletakkan

Ini keputusan arsitektural terpenting pada pertemuan ini, dan jawabannya bukan "salah satu", melainkan "dua-duanya, pada tempat yang berbeda".

	Middleware	Service
Menjawab	Boleh mengakses jenis endpoint ini?	Boleh mengakses baris data yang ini?
Butuh data?	Tidak	Ya — perlu tahu data milik siapa
Contoh	GET /users (melihat daftar semua user)	GET /users/:id (miliknya sendiri atau bukan)
Keuntungan	Terlihat jelas di file route, satu baris per endpoint	Dapat mempertimbangkan isi data
Risiko	Lupa memasang = endpoint terbuka	Tersebar di banyak method = mudah terlewat

Aturan praktisnya: **bila keputusan dapat diambil tanpa menyentuh database, letakkan di middleware; bila perlu tahu data itu milik siapa, letakkan di service.** Meletakkan semuanya di middleware memaksa middleware melakukan query, dan itu berarti middleware mulai mengenal struktur database — persis yang dihindari Clean Architecture pada Modul 4.

A.7 401 dan 403

Status	Makna	Situasi
401 Unauthorized	Identitas belum diketahui	Token tidak ada, kadaluarsa, atau tidak sah
403 Forbidden	Identitas diketahui, tetapi haknya kurang	Token sah, namun role-nya tidak memiliki permission yang diminta

Nama 401 memang menyesatkan — seharusnya "Unauthenticated". Cara mengingatnya: **401 artinya "saya tidak kenal Anda, silakan login"**, sedangkan **403 artinya "saya kenal Anda, dan jawabannya tetap tidak"**. Mengirim 401 padahal seharusnya 403 membuat frontend melakukan hal yang salah: ia akan melempar pengguna ke halaman login, padahal login ulang tidak akan mengubah apa pun.

A.8 Broken Access Control

Pada OWASP Top 10 edisi 2021, Broken Access Control menempati peringkat pertama — naik dari peringkat kelima pada edisi 2017. Bentuk kegagalannya yang paling sering muncul justru sangat sederhana:

- Mengandalkan frontend untuk menyembunyikan tombol, tanpa pemeriksaan di backend. Menyembunyikan tombol hanya menyembunyikan tombol; endpoint-nya tetap dapat dipanggil langsung.
- IDOR (Insecure Direct Object Reference): endpoint menerima :id dan langsung mengambil datanya tanpa bertanya apakah pemanggil berhak atas id tersebut. Mengganti /users/7 menjadi /users/8 sudah cukup.
- Mass assignment: menerima seluruh isi body JSON apa adanya, sehingga penyerang cukup menambahkan "role":"admin" pada request pendaftaran. Kita sudah mencegah ini sejak Modul 5 dengan memisahkan struct request dari struct entity.

- Endpoint baru yang lupa dipasang pemeriksaan. Ini yang paling sering terjadi di dunia nyata, dan justru inilah alasan kita meletakkan pemeriksaan pada file route: agar sebuah endpoint tanpa penjaga terlihat mencolok saat dibaca.

A.9 Keterbatasan Role di Dalam Token

Role kita dibawa di dalam JWT, dan JWT bersifat *stateless* — isinya tidak diperiksa ulang ke database pada setiap request. Itu cepat, tetapi menimbulkan satu konsekuensi yang wajib Anda sadari: **perubahan role tidak berlaku seketika**.

Bila admin menurunkan role seseorang dari staff menjadi user, token lama orang tersebut masih berisi "role":"staff" dan masih berlaku sampai kadaluarsa. Inilah alasan access token diberi umur pendek (15 menit pada konfigurasi kita): masa berlaku itu sekaligus batas atas keterlambatan pencabutan hak.

Bila keterlambatan 15 menit tidak dapat diterima — misalnya pada sistem keuangan — pilihannya adalah memeriksa role ke database pada setiap request (kehilangan keuntungan *stateless*), atau menyimpan daftar token yang dicabut. Keduanya menukar kecepatan dengan kesegaran data. Tidak ada pilihan yang gratis.

B. Langkah Panduan

Lanjutkan proyek latihan-fiber dari Modul 5. Tidak ada dependency baru pada pertemuan ini — seluruh perubahan berada pada kode dan skema database kita sendiri.

LANGKAH 1 — Membuat Skema RBAC di Database

Hak akses disimpan di database, bukan ditulis keras di dalam kode. Alasannya: kebijakan akses berubah lebih sering daripada kode, dan mengubah kebijakan seharusnya tidak menuntut compile ulang.

► migrations/003_rbac.sql

```
-- -----
-- roles – daftar role yang diakui sistem
-- -----
CREATE TABLE IF NOT EXISTS roles (
  name          VARCHAR(20) PRIMARY KEY,
  description    VARCHAR(150) NOT NULL,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

INSERT INTO roles (name, description) VALUES
  ('admin', 'Akses penuh terhadap seluruh data dan pengaturan'),
  ('staff', 'Boleh melihat data seluruh user, tetapi tidak boleh mengubah'),
  ('user', 'Hanya boleh mengelola datanya sendiri')
ON CONFLICT (name) DO NOTHING;

-- -----
-- permissions – daftar tindakan yang dapat diberikan kepada role
-- -----
CREATE TABLE IF NOT EXISTS permissions (
  name          VARCHAR(50) PRIMARY KEY,
  description    VARCHAR(150) NOT NULL
);

INSERT INTO permissions (name, description) VALUES
  ('user:list', 'Melihat daftar seluruh user'),
  ('user:read:any', 'Melihat data user mana pun'),
  ('user:update:any', 'Mengubah data user mana pun'),
  ('user:delete', 'Menghapus user'),
  ('role:assign', 'Mengubah role milik user lain')
```

```

ON CONFLICT (name) DO NOTHING;

-----
-- role_permissions - tabel penghubung, inti dari model RBAC
-----
CREATE TABLE IF NOT EXISTS role_permissions (
    role_name      VARCHAR(20) NOT NULL
    REFERENCES roles(name)      ON DELETE CASCADE,
    permission_name VARCHAR(50) NOT NULL
    REFERENCES permissions(name) ON DELETE CASCADE,
    PRIMARY KEY (role_name, permission_name)
);

INSERT INTO role_permissions (role_name, permission_name) VALUES
    ('admin', 'user:list'),
    ('admin', 'user:read:any'),
    ('admin', 'user:update:any'),
    ('admin', 'user:delete'),
    ('admin', 'role:assign'),
    ('staff', 'user:list'),
    ('staff', 'user:read:any')
ON CONFLICT DO NOTHING;
-- role 'user' sengaja tidak diberi permission apa pun.

-----
-- Kunci column role pada users agar hanya berisi role yang dikenal.
-- Sebelumnya column ini VARCHAR biasa: apa pun bisa masuk.
-----
UPDATE users SET role = 'user' WHERE role NOT IN (SELECT name FROM roles);

ALTER TABLE users DROP CONSTRAINT IF EXISTS users_role_fkey;
ALTER TABLE users
    ADD CONSTRAINT users_role_fkey
    FOREIGN KEY (role) REFERENCES roles(name) ON UPDATE CASCADE;

CREATE INDEX IF NOT EXISTS users_role_idx ON users (role);

```

Tiga keputusan yang perlu dipahami, bukan sekadar disalin:

- **PRIMARY KEY gabungan** pada `role_permissions` mencegah satu role menerima permission yang sama dua kali. Aturan itu dijaga database, bukan diingat-ingat oleh kode.
- **FOREIGN KEY pada `users.role`** memastikan tidak ada user dengan role yang tidak dikenal. Tanpa constraint ini, satu baris hasil INSERT manual yang salah ketik akan menghasilkan user yang selamanya ditolak semua endpoint, dan penyebabnya sangat sulit ditemukan.
- **Baris UPDATE sebelum ALTER TABLE** wajib ada. Data lama Anda mungkin berisi role yang belum terdaftar, dan penambahan FOREIGN KEY akan gagal bila masih ada baris yang melanggar. Migration yang menghapus atau mengubah data harus selalu Anda baca dua kali sebelum dijalankan pada data sungguhan.

Jalankan:

```
psql -U postgres -d praktikum_backend -f migrations/003_rbac.sql
```

LANGKAH 2 — PermissionSet — Menyimpan Hak Akses di Memori

Membaca tabel `role_permissions` pada setiap request berarti satu query tambahan untuk setiap request, padahal isinya nyaris tidak pernah berubah. Kita membacanya sekali saat aplikasi menyala, lalu menyimpannya dalam bentuk yang cepat dicari.

► [helper/authz.go](#)

```
package helper
```

```

import "sort"

// PermissionSet menyimpan pemetaan role ke daftar permission miliknya.
type PermissionSet struct {
    byRole map[string]map[string]struct{}
}

// NewPermissionSet mengubah hasil query menjadi bentuk yang cepat dicari.
// map bersarang dipilih karena pencarian "apakah role X punya permission Y"
// menjadi satu langkah, bukan perulangan sepanjang daftar.
func NewPermissionSet(raw map[string][]string) *PermissionSet {
    byRole := make(map[string]map[string]struct{}, len(raw))

    for role, permissions := range raw {
        set := make(map[string]struct{}, len(permissions))
        for _, permission := range permissions {
            set[permission] = struct{}{}
        }
        byRole[role] = set
    }

    return &PermissionSet{byRole: byRole}
}

// Can menjawab satu pertanyaan: apakah role ini memiliki permission itu.
//
// Perhatikan perilakunya ketika role atau permission tidak dikenal:
// jawabannya SELALU false. Prinsipnya disebut fail closed.
func (p *PermissionSet) Can(role, permission string) bool {
    if p == nil {
        return false
    }

    permissions, ok := p.byRole[role]
    if !ok {
        return false
    }

    _, granted := permissions[permission]
    return granted
}

// PermissionsOf mengembalikan seluruh permission milik sebuah role,
// terurut, untuk ditampilkan pada endpoint profil.
func (p *PermissionSet) PermissionsOf(role string) []string {
    result := []string{}
    if p == nil {
        return result
    }

    for permission := range p.byRole[role] {
        result = append(result, permission)
    }
    sort.Strings(result)
    return result
}

// KnownRoles mengembalikan daftar role yang dikenal sistem, terurut.
func (p *PermissionSet) KnownRoles() []string {
    result := []string{}
    if p == nil {
        return result
    }

    for role := range p.byRole {
        result = append(result, role)
    }
    sort.Strings(result)
    return result
}

```

```
// IsKnownRole dipakai saat memvalidasi permintaan pengantian role.
func (p *PermissionSet) IsKnownRole(role string) bool {
    if p == nil {
        return false
    }
    _, ok := p.byRole[role]
    return ok
}
```

Mengapa `map[string]struct{}` dan bukan `[]string`? Karena pencarian pada slice berarti menelusuri seluruh isinya, sedangkan pencarian pada map langsung menuju sasaran. `struct{}` dipakai sebagai nilai karena ukurannya nol byte — kita hanya butuh keberadaan kunci, bukan nilainya. Pola ini adalah cara idiomatik membuat himpunan (set) di Go, yang tidak memiliki tipe set bawaan.

Mengapa pemeriksaan nil ada di setiap method

Sekilas pemeriksaan `p == nil` tampak berlebihan. Ia ada justru karena prinsip fail closed. Bila suatu saat ada jalur kode yang lupa menginisialisasi `PermissionSet`, tanpa pemeriksaan ini program akan panic dan mati; dengan pemeriksaan ini program menolak seluruh akses. Menolak semua akses adalah kegagalan yang aman. Mengizinkan semua akses tidak.

LANGKAH 3 — Membaca Pemetaan Role dari Database

► `app/repository/role_repository.go`

```
package repository

import (
    "context"
    "fmt"

    "github.com/jackc/pgx/v5/pgxpool"
)

// RoleRepository membaca pemetaan role ke permission dari database.
type RoleRepository interface {
    LoadPermissions(ctx context.Context) (map[string][]string, error)
}

type rolePostgresRepository struct{ pool *pgxpool.Pool }

func NewRoleRepository(pool *pgxpool.Pool) RoleRepository {
    return &rolePostgresRepository{pool: pool}
}

// LoadPermissions mengambil SELURUH pasangan role dan permission sekaligus.
//
// LEFT JOIN dipakai dengan sengaja: role yang belum punya permission apa pun
// tetap harus muncul, supaya sistem tahu role tersebut memang dikenal
// meskipun daftar permission-nya kosong.
func (r *rolePostgresRepository) LoadPermissions(
    ctx context.Context,
) (map[string][]string, error) {
    rows, err := r.pool.Query(ctx,
        `SELECT r.name, COALESCE(rp.permission_name, '')
        FROM roles r
        LEFT JOIN role_permissions rp ON rp.role_name = r.name
        ORDER BY r.name, rp.permission_name`)
    if err != nil {
        return nil, fmt.Errorf("mengambil permission: %w", err)
    }
    defer rows.Close()
```

```

result := map[string][]string{}
for rows.Next() {
    var role, permission string
    if err := rows.Scan(&role, &permission); err != nil {
        return nil, fmt.Errorf("membaca row permission: %w", err)
    }

    if _, ok := result[role]; !ok {
        result[role] = []string{}
    }
    if permission != "" {
        result[role] = append(result[role], permission)
    }
}
if err := rows.Err(); err != nil {
    return nil, fmt.Errorf("membaca hasil query permission: %w", err)
}

return result, nil
}

```

LEFT JOIN di sini bukan pilihan gaya. Dengan INNER JOIN, role user — yang memang tidak punya permission apa pun — tidak akan muncul sama sekali pada hasil query. Akibatnya `IsKnownRole("user")` akan mengembalikan false, dan admin tidak akan pernah bisa menetapkan role user kepada siapa pun. Satu kata kunci SQL yang salah menghasilkan bug yang baru terasa berminggu-minggu kemudian.

COALESCE dipakai karena LEFT JOIN menghasilkan NULL untuk role tanpa pasangan. Mengubahnya menjadi string kosong membuat Scan tidak perlu memakai tipe pointer, dan baris kosong itu kemudian kita saring di dalam perulangan.

LANGKAH 4 — Memuat Permission Sekali Saat Aplikasi Menyala

► main.go – tambahan di antara repository dan service

```

userRepository := repository.NewUserRepository(pool)
tokenRepository := repository.NewTokenRepository(pool)
roleRepository := repository.NewRoleRepository(pool)

// Pemetaan role ke permission dibaca SEKALI saat aplikasi menyala.
// Konsekuensinya: perubahan hak akses di database baru berlaku setelah
// aplikasi dijalankan ulang. Itu keputusan sadar, bukan kelalaian.
rawPermissions, err := roleRepository.LoadPermissions(context.Background())
if err != nil {
    logger.Error("gagal memuat permission", slog.String("error", err.Error()))
    os.Exit(1)
}
permissions := helper.NewPermissionSet(rawPermissions)
logger.Info("permission dimuat", slog.Any("roles", permissions.KnownRoles()))

userService := service.NewUserService(userRepository, permissions)
authService := service.NewAuthService(
    userRepository, tokenRepository, jwtManager, permissions,
    time.Duration(config.GetEnvInt("JWT_REFRESH_TTL_DAYS", 7))*24*time.Hour,
)

app := config.NewApp(logger, route.Dependencies{
    Pool:      pool,
    JWT:       jwtManager,
    Permissions: permissions,
    UserService: userService,
    AuthService: authService,
})

```

Perhatikan `os.Exit(1)` ketika pemuatan gagal. Aplikasi sengaja menolak menyala daripada berjalan dengan `PermissionSet` kosong. Dengan `PermissionSet` kosong, `Can()` akan selalu mengembalikan `false` dan seluruh endpoint akan membalas 403 — keadaan yang membingungkan dan sulit didiagnosis. Lebih baik gagal keras di detik pertama daripada gagal diam-diam sepanjang hari.

Baris `logger.Info("permission dimuat", ...)` juga bukan hiasan. Saat aplikasi menyala Anda akan melihat:

```
{"time":"...", "level":"INFO", "msg":"permission dimuat",  
  "roles":["admin", "staff", "user"]}
```

Bila baris itu menampilkan daftar role yang tidak Anda harapkan, Anda tahu masalahnya ada pada migration — sebelum satu request pun masuk.

LANGKAH 5 — Middleware RequirePermission

► `middleware/authz.go`

```
package middleware

import (
    "github.com/gofiber/fiber/v2"

    "latihan-fiber/helper"
)

// RequirePermission menolak request yang role-nya tidak memiliki
// permission tertentu. Dipasang pada route yang haknya dapat diputuskan
// TANPA melihat isi data — misalnya "boleh melihat daftar seluruh user".
//
// Untuk keputusan yang bergantung pada isi data (misalnya "boleh mengubah
// data ini karena miliknya sendiri"), pemeriksaan tidak bisa di sini:
// middleware belum tahu data siapa yang akan disentuh. Pemeriksaan
// semacam itu berada di layer service.
func RequirePermission(perms *helper.PermissionSet, permission string) fiber.Handler {
    return func(c *fiber.Ctx) error {
        user, ok := helper.CurrentUser(c)
        if !ok {
            // Sampai di sini tanpa identitas berarti RequireAuth belum
            // dipasang. Tolak, jangan diloloskan.
            return helper.Fail(c, fiber.StatusUnauthorized, "belum terautentikasi")
        }

        if !perms.Can(user.Role, permission) {
            return helper.Fail(c, fiber.StatusForbidden,
                "role "+user.Role+" tidak memiliki hak "+permission)
        }

        return c.Next()
    }
}

// RequireRole memeriksa nama role secara langsung.
//
// Cara ini lebih sederhana, tetapi lebih kaku: menambah satu role baru
// berarti menyunting seluruh route yang menyebut nama role lama.
// Disediakan di sini sebagai pembanding, bukan sebagai pilihan utama.
func RequireRole(roles ...string) fiber.Handler {
    allowed := make(map[string]struct{}, len(roles))
    for _, role := range roles {
        allowed[role] = struct{}{}
    }

    return func(c *fiber.Ctx) error {
        user, ok := helper.CurrentUser(c)
        if !ok {
            return helper.Fail(c, fiber.StatusUnauthorized, "belum terautentikasi")
        }
    }
}
```

```

    }

    if _, granted := allowed[user.Role]; !granted {
        return helper.Fail(c, fiber.StatusForbidden,
            "role Anda tidak berhak mengakses endpoint ini")
    }

    return c.Next()
}
}

```

Mengapa dua fungsi, padahal hanya satu yang dipakai

RequireRole sengaja disertakan sebagai pembanding, karena inilah bentuk yang paling sering Anda temui di tutorial. Bandingkan apa yang terjadi ketika kebijakan berubah:

Skenario	Dengan RequireRole	Dengan RequirePermission
Muncul role baru "auditor" yang boleh melihat daftar user	Sunting setiap baris route yang menyebut role, compile ulang, deploy ulang	Tambah dua baris INSERT di database, jalankan ulang aplikasi
Membaca file route untuk tahu siapa boleh apa	Terbaca jelas: nama role langsung tertulis	Perlu melihat tabel role_permissions untuk tahu role mana yang punya permission itu
Salah ketik nama	Role tidak dikenal, akses ditolak	Permission tidak dikenal, akses ditolak

Jadi **RequirePermission tidak unggul dalam segala hal** — ia menukar keterbacaan langsung dengan keluwesan. Untuk aplikasi kecil dengan dua role yang tidak akan pernah bertambah, RequireRole adalah pilihan yang wajar dan lebih jujur. Kita memilih RequirePermission karena mata kuliah ini membahas sistem yang akan tumbuh.

LANGKAH 6 — File Route sebagai Peta Hak Akses

Satu perubahan kecil dengan dampak besar: mulai sekarang file route dapat dibaca sebagai daftar hak akses. Siapa pun yang ingin tahu "endpoint mana yang dijaga apa" cukup membuka satu file.

► route/route.go

```

type Dependencies struct {
    Pool      *pgxpool.Pool
    JWT       *helper.JWTManager
    Permissions *helper.PermissionSet
    UserService *service.UserService
    AuthService *service.AuthService
}

func Register(app *fiber.App, deps Dependencies) {
    api := app.Group("/api/v1")

    // --- publik ---
    api.Get("/health", healthCheck(deps.Pool))

    // --- autentikasi ---
    auth := api.Group("/auth", middleware.RequireJSON)
    auth.Post("/register", deps.AuthService.Register)
    auth.Post("/login", middleware.LoginRateLimiter(), deps.AuthService.Login)
    auth.Post("/refresh", deps.AuthService.Refresh)
    auth.Post("/logout", deps.AuthService.Logout)
    auth.Get("/me", middleware.RequireAuth(deps.JWT), deps.AuthService.Me)
}

```

```
// --- wajib login, hak akses diperiksa per endpoint ---
users := api.Group("/users",
    middleware.RequireJSON,
    middleware.RequireAuth(deps.JWT))

perms := deps.Permissions

// Hak dapat diputuskan tanpa melihat data -> middleware.
users.Get("/",
    middleware.RequirePermission(perms, "user:list"),
    deps.UserService.List)
users.Post("/",
    middleware.RequirePermission(perms, "user:update:any"),
    deps.UserService.Create)
users.Delete("/:id",
    middleware.RequirePermission(perms, "user:delete"),
    deps.UserService.Delete)
users.Patch("/:id/role",
    middleware.RequirePermission(perms, "role:assign"),
    deps.UserService.AssignRole)

// Hak bergantung pada kepemilikan data -> diperiksa di service.
users.Get("/:id", deps.UserService.Get)
users.Put("/:id", deps.UserService.Replace)
users.Patch("/:id", deps.UserService.Patch)
}
```

Tiga baris terakhir tampak tidak dijaga — apakah itu bug?

Bukan. Ketiganya berada di dalam group users, sehingga tetap melewati RequireAuth: tanpa token, jawabannya 401. Yang tidak dapat diputuskan di sini adalah apakah pemanggil berhak atas baris data tertentu, karena middleware belum tahu id yang diminta merujuk data milik siapa.

Meski begitu, kekhawatiran Anda beralasan dan patut dipelihara. Endpoint yang terlihat tanpa penjaga harus selalu memicu pertanyaan. Karena itu komentar dua baris di atasnya bukan basa-basi: ia menjelaskan kepada pembaca berikutnya bahwa penjaganya ada, hanya berada di tempat lain.

LANGKAH 7 — Pemeriksaan Kepemilikan di Layer Service

Sekarang bagian yang tidak dapat dikerjakan middleware. Kita letakkan aturannya di file terpisah yang tidak menyentuh Fiber maupun database, sehingga dapat diuji hanya dengan memanggilnya.

► `app/service/authz_rules.go`

```
package service

import (
    "strings"

    "latihan-fiber/app/model"
    "latihan-fiber/helper"
)

// CanAccessUser memutuskan apakah seseorang boleh menyentuh data user lain.
//
// Dua jalur yang diizinkan:
// 1. Kepemilikan (ownership) — data itu miliknya sendiri.
// 2. Permission — role-nya memang berhak atas data siapa pun.
//
// Urutannya disengaja: pemeriksaan kepemilikan didahulukan karena paling
// murah dan paling sering benar. Bila keduanya gagal, jawabannya false.
func CanAccessUser(
    current model.AuthUser,
    targetID int,
    perms *helper.PermissionSet,
    anyPermission string,
```

```

) bool {
    if current.UserID == targetID {
        return true
    }
    return perms.Can(current.Role, anyPermission)
}

// ValidateAssignRole memeriksa permintaan pergantian role.
//
// Perhatikan aturan terakhir: seseorang tidak boleh mengubah role dirinya
// sendiri. Tanpa aturan itu, satu-satunya admin dapat menurunkan dirinya
// sendiri menjadi user biasa dan sistem kehilangan admin selamanya.
func ValidateAssignRole(
    current model.AuthUser,
    targetID int,
    req model.AssignRoleRequest,
    perms *helper.PermissionSet,
) map[string]string {
    errs := map[string]string{}

    role := strings.TrimSpace(req.Role)
    if role == "" {
        errs["role"] = "wajib diisi"
        return errs
    }

    if !perms.IsKnownRole(role) {
        errs["role"] = "role tidak dikenal, pilih salah satu dari: " +
            strings.Join(perms.KnownRoles(), ", ")
    }

    if current.UserID == targetID {
        errs["role"] = "tidak boleh mengubah role diri sendiri"
    }

    return errs
}

```

Memakainya di user_service.go

► app/service/user_service.go – bagian yang berubah

```

type UserService struct {
    repo repository.UserRepository
    perms *helper.PermissionSet
}

func NewUserService(
    repo repository.UserRepository,
    perms *helper.PermissionSet,
) *UserService {
    return &UserService{repo: repo, perms: perms}
}

// ----- GET /users/:id -----
func (s *UserService) Get(c *fiber.Ctx) error {
    ctx, cancel := helper.RequestContext(c)
    defer cancel()

    current, ok := helper.CurrentUser(c)
    if !ok {
        return helper.Fail(c, fiber.StatusUnauthorized, "belum terautentikasi")
    }

    id, valid := helper.ParamID(c)
    if !valid {
        return helper.Fail(c, fiber.StatusBadRequest, "id harus berupa angka positif")
    }

    // Pemeriksaan hak akses dilakukan SEBELUM data diambil.
    // Bila dibalik, penyerang tetap dapat menyimpulkan keberadaan sebuah id

```

```
// dari perbedaan waktu tanggap antara 403 dan 404.
if !CanAccessUser(current, id, s.perms, "user:read:any") {
    return helper.Fail(c, fiber.StatusForbidden,
        "tidak berhak mengakses data user lain")
}

user, err := s.repo.FindByID(ctx, id)
if err != nil {
    return translateError(c, err, "gagal mengambil data user")
}

return helper.Success(c, fiber.StatusOK, "user ditemukan", user)
}
```

Terapkan pola yang sama pada Replace dan Patch, dengan permission `user:update:any`.

Mengapa urutan "periksa hak, baru ambil data" itu penting

Bila urutannya dibalik — ambil data dulu, periksa hak kemudian — request untuk id yang tidak ada akan melewati satu query database, sedangkan request untuk id yang ada tetapi bukan miliknya juga melewati query yang sama namun berakhir 403. Selisih waktu tanggap di antara keduanya kecil, tetapi terukur, dan cukup untuk memetakan id mana saja yang ada di dalam sistem.

Serangan semacam ini disebut timing attack. Ia tidak membocorkan isi data, hanya keberadaannya — dan pada banyak sistem, keberadaan data sudah merupakan informasi yang tidak boleh bocor.

Endpoint baru: mengubah role

► `app/service/user_service.go` — method baru

```
// ----- PATCH /users/:id/role -----
// Dijaga middleware dengan permission role:assign.
func (s *UserService) AssignRole(c *fiber.Ctx) error {
    ctx, cancel := helper.RequestContext(c)
    defer cancel()

    current, ok := helper.CurrentUser(c)
    if !ok {
        return helper.Fail(c, fiber.StatusUnauthorized, "belum terautentikasi")
    }

    id, valid := helper.ParamID(c)
    if !valid {
        return helper.Fail(c, fiber.StatusBadRequest, "id harus berupa angka positif")
    }

    var req model.AssignRoleRequest
    if err := c.BodyParser(&req); err != nil {
        return helper.Fail(c, fiber.StatusBadRequest, "body harus berupa JSON yang valid")
    }

    if errs := ValidateAssignRole(current, id, req, s.perms); len(errs) > 0 {
        return helper.FailValidation(c, errs)
    }

    result, err := s.repo.UpdateRole(ctx, id, strings.TrimSpace(req.Role))
    if err != nil {
        return translateError(c, err, "gagal mengubah role user")
    }

    return helper.Success(c, fiber.StatusOK, "role user berhasil diubah", result)
}

// ----- DELETE /users/:id -----
func (s *UserService) Delete(c *fiber.Ctx) error {
```

```
// ... pengambilan current dan id seperti biasa ...

// Punya permission menghapus tidak berarti boleh menghapus dirinya sendiri.
if current.UserID == id {
    return helper.Fail(c, fiber.StatusForbidden,
        "tidak boleh menghapus akun sendiri")
}

if err := s.repo.Delete(ctx, id); err != nil {
    return translateError(c, err, "gagal menghapus user")
}

return helper.NoContent(c)
}
```

► app/model/user.go – struct baru

```
// AssignRoleRequest dipakai endpoint PATCH /users/:id/role.
type AssignRoleRequest struct {
    Role string `json:"role"`
}
```

► app/repository/user_repository.go – method baru

```
// UpdateRole sengaja dipisah dari Update. Mengubah role adalah tindakan
// istimewa yang dijaga permission tersendiri, sehingga tidak boleh ikut
// terbawa oleh endpoint perubahan data biasa.
func (r *userPostgresRepository) UpdateRole(
    ctx context.Context, id int, role string,
) (model.User, error) {
    updated, err := scanUser(r.pool.QueryRow(ctx,
        "UPDATE users SET role = $1 WHERE id = $2 RETURNING "+userColumns,
        role, id))
    if err != nil {
        if errors.Is(err, pgx.ErrNoRows) {
            return model.User{}, ErrNotFound
        }
        return model.User{}, fmt.Errorf("mengubah role user: %w", err)
    }
    return updated, nil
}
```

Dua aturan pada langkah ini pantas diperhatikan, karena keduanya melindungi sistem dari dirinya sendiri:

- **Tidak boleh mengubah role diri sendiri.** Tanpa aturan ini, satu-satunya admin dapat menurunkan dirinya menjadi user biasa. Setelah itu tidak ada lagi yang punya permission role:assign, dan sistem kehilangan admin selamanya — hanya dapat dipulihkan lewat akses langsung ke database.
- **Tidak boleh menghapus akun sendiri.** Perhatikan bahwa aturan ini berada di service, bukan di middleware. Middleware hanya tahu "role ini punya permission user:delete"; ia tidak tahu bahwa id yang hendak dihapus kebetulan adalah id pemanggilnya sendiri. Punya izin tidak sama dengan boleh melakukannya dalam segala keadaan.

LANGKAH 8 — Membuat Keputusan Akses Dapat Ditelusuri

Sistem authorization yang tidak meninggalkan jejak sulit dipercaya. Ketika seseorang melapor "saya tidak bisa mengakses halaman X", Anda perlu tahu siapa dia, perannya apa, dan permission mana yang kurang.

► middleware/middleware.go – RequestLogger

```
attrs := []any{
    slog.String("request_id", requestID),
    slog.String("method", c.Method()),
    slog.String("path", c.Path()),
    slog.Int("status", c.Response().StatusCode()),
}
```

```

    slog.Duration("duration", time.Since(start)),
    slog.String("ip", c.IP()),
}

// Identitas ikut dicatat bila request sudah melewati RequireAuth.
// Tanpa ini, log sebuah 403 tidak berguna: kita tahu ada yang ditolak,
// tetapi tidak tahu siapa dan mengapa.
if user, ok := helper.CurrentUser(c); ok {
    attrs = append(attrs,
        slog.Int("user_id", user.UserID),
        slog.String("role", user.Role))
}

logger.Info("http_request", attrs...)

```

Hasilnya, sebuah penolakan kini terbaca lengkap:

```

{"level":"INFO","msg":"http_request","method":"GET","path":"/api/v1/users",
 "status":403,"ip":"127.0.0.1","user_id":3,"role":"user"}

```

Menampilkan permission pada endpoint profil

Frontend perlu tahu tombol apa saja yang layak ditampilkan. Karena itu `/auth/me` sekarang menyertakan daftar permission:

► `app/service/auth_service.go` – method `Me`

```

return helper.Success(c, fiber.StatusOK, "profil berhasil diambil", fiber.Map{
    "user":      user,
    "permissions": s.perms.PermissionsOf(user.Role),
})

```

Ini kemudahan tampilan, bukan pengamanan

Daftar permission yang dikirim ke frontend hanya membantu menyembunyikan tombol yang tidak berguna. Ia tidak mengamankan apa pun: siapa pun dapat memanggil endpoint langsung tanpa melewati frontend sama sekali.

Pengamanan yang sesungguhnya tetap berada di middleware dan service. Bila suatu saat Anda tergoda menghapus pemeriksaan di backend karena "kan tombolnya sudah disembunyikan", ingat bahwa itu persis definisi Broken Access Control pada OWASP Top 10.

LANGKAH 9 — Menguji Matriks Hak Akses

Kode authorization adalah bagian yang paling berbahaya bila salah, dan paling mudah tampak benar padahal tidak. Karena itu ia harus diuji secara sistematis, bukan dicoba seadanya.

Siapkan tiga akun, lalu naikan dua di antaranya lewat database:

```

# daftar tiga akun
curl -X POST localhost:3000/api/v1/auth/register \
-H 'Content-Type: application/json' \
-d '{"username":"admin","email":"admin@unair.ac.id","password":"Rahasia123!"}'

# ulangi untuk staff dan biasa

# naikan role lewat psql
psql -U postgres -d praktikum_backend -c \
"UPDATE users SET role='admin' WHERE username='admin';
 UPDATE users SET role='staff' WHERE username='staff';"

# ambil token
curl -s -X POST localhost:3000/api/v1/auth/login \
-H 'Content-Type: application/json' \

```

```
-d '{"username":"admin","password":"Rahasia123!"}'
```

Kemudian telusuri seluruh matriks. Hasil yang benar adalah sebagai berikut — bukan perkiraan, melainkan keluaran yang sudah diverifikasi:

Request	admin	staff	user
GET /users	200	200	403
GET /users/:id — dirinya sendiri	200	200	200
GET /users/:id — milik orang lain	200	200	403
PUT /users/:id — milik orang lain	200	403	403
DELETE /users/:id — orang lain	204	403	403
DELETE /users/:id — dirinya sendiri	403	403	403
PATCH /users/:id/role — orang lain	200	403	403
PATCH /users/:id/role — dirinya sendiri	422	403	403
Tanpa Authorization header	401	401	401

Perhatikan baris "PATCH role — dirinya sendiri" pada kolom admin: jawabannya 422, bukan 403. Admin memang memiliki permission role:assign — haknya tidak kurang — tetapi permintaannya melanggar aturan bisnis. Membedakan "hak Anda kurang" (403) dari "permintaan Anda tidak masuk akal" (422) membuat pesan kesalahan jauh lebih menolong bagi pemakai API.

Satu pengujian yang hampir selalu terlewat

Ubah role seseorang, lalu panggil endpoint memakai token lamanya — token yang diterbitkan sebelum perubahan:

```
# user id 3 dinaikkan menjadi staff oleh admin
curl -X PATCH localhost:3000/api/v1/users/3/role \
  -H "Authorization: Bearer $TOKEN_ADMIN" \
  -H 'Content-Type: application/json' -d '{"role":"staff"}'
# -> 200

# tetapi token LAMA milik user 3 masih membawa role "user"
curl localhost:3000/api/v1/users -H "Authorization: Bearer $TOKEN_LAMA"
# -> 403 {"message":"role user tidak memiliki hak user:list"}

# setelah login ulang, token baru membawa role "staff"
curl localhost:3000/api/v1/users -H "Authorization: Bearer $TOKEN_BARU"
# -> 200
```

Inilah wujud nyata dari A.9. Perilaku ini bukan bug, melainkan harga yang kita bayar untuk token yang stateless. Yang menjadikannya bug adalah bila Anda tidak menyadarinya — misalnya saat mencabut hak seorang pegawai yang baru diberhentikan, lalu mengira pencabutan itu berlaku seketika.

- ? Apa yang terjadi bila urutan middleware dibalik: RequirePermission dipasang sebelum RequireAuth? Coba, lalu jelaskan mengapa jawabannya 401 dan bukan 403.
- ? Hapus satu baris pada tabel role_permissions milik admin, lalu panggil endpoint terkait tanpa menjalankan ulang aplikasi. Mengapa hasilnya belum berubah?
- ? Tambahkan role "auditor" yang hanya memiliki user:list. Berapa baris kode Go yang perlu Anda ubah? Bandingkan dengan bila Anda memakai RequireRole.
- ? Salah ketik nama permission pada file route, misalnya "user:lis". Apa yang terjadi, dan bagaimana Anda mengetahuinya? Bandingkan dengan yang akan terjadi bila Can() dirancang fail open.
- ? Pindahkan pemeriksaan CanAccessUser ke posisi setelah repo.FindByID. Endpoint tetap berfungsi — jadi apa sebenarnya yang hilang?

C. Tugas Mandiri

Lanjutkan entity students yang Anda kembangkan sejak Modul 3. Sampai sekarang, seluruh endpoint students dapat diakses siapa saja yang berhasil login. Tugas Anda adalah menutup lubang itu.

C.1 Skema dan Kebijakan Akses

Buat file migrations/004_student_permissions.sql yang menambahkan permission berikut dan memasangkannya pada role yang sesuai:

Permission	Diberikan kepada
student:list	admin, staff
student:read:any	admin, staff
student:create	admin, staff
student:update:any	admin
student:delete	admin

Tambahkan pula column owner_id INTEGER REFERENCES users(id) pada tabel students, yang menandai user mana yang mendaftarkan data tersebut. Column inilah yang nanti menjadi dasar pemeriksaan kepemilikan.

Perhatian: tabel students Anda mungkin sudah berisi data. Pikirkan apa yang harus terjadi pada baris-baris lama yang belum punya owner_id sebelum Anda menambahkan constraint. Migration yang gagal di tengah jalan lebih merepotkan daripada migration yang ditulis hati-hati sejak awal.

C.2 Implementasi

1. Pasang RequirePermission pada endpoint students yang haknya dapat diputuskan tanpa melihat data: GET /students, POST /students, dan DELETE /students/:id.
2. Buat file app/service/student_authz_rules.go berisi fungsi murni CanAccessStudent(current model.AuthUser, ownerID int, perms *helper.PermissionSet, anyPermission string) bool. Fungsi ini tidak boleh mengimpor fiber maupun repository.

3. Terapkan pemeriksaan kepemilikan pada GET /students/:id, PUT /students/:id, dan PATCH /students/:id: pemilik data (owner_id sama dengan id pemanggil) selalu boleh; selain itu diperlukan permission :any yang sesuai.
4. Pastikan owner_id diisi otomatis dari identitas pemanggil saat POST /students, dan pastikan column tersebut TIDAK dapat diubah lewat body request. Jelaskan pada laporan mengapa hal ini penting.
5. Catat user_id dan role pada log setiap request students, mengikuti pola Langkah 8.

C.3 Bukti Pengujian

Susun tabel matriks hak akses untuk seluruh endpoint students, dengan kolom untuk role admin, staff, dan user (sebagai pemilik data maupun bukan pemilik). Untuk setiap sel, sertakan perintah curl dan status code yang benar-benar Anda peroleh.

Sertakan pula minimal dua **pengujian negatif**: percobaan yang seharusnya gagal, beserta bukti bahwa ia memang gagal. Contoh yang layak dicoba: mengirim "owner_id": 1 di dalam body POST /students untuk mengaku-aku sebagai pemilik lain, dan memanggil PUT /students/:id atas data milik orang lain memakai token role user.

C.4 Analisis Singkat

Jawab dalam laporan, masing-masing satu paragraf:

1. Mengapa pemeriksaan kepemilikan tidak dapat dipindahkan ke middleware? Jawab dengan merujuk pada kode Anda sendiri, bukan secara umum.
2. Sistem Anda kini punya dua tempat pemeriksaan akses: route dan service. Sebutkan satu risiko konkret dari pembagian ini, dan bagaimana Anda mengurangi risiko tersebut.
3. Bila suatu hari muncul kebutuhan "dosen wali hanya boleh melihat mahasiswa bimbingannya", apakah RBAC masih memadai? Bila tidak, apa yang perlu ditambahkan?

Ketentuan Pengumpulan

Nama berkas	Tugas6_NIM.pdf
Isi laporan	Potongan kode penting (migration, middleware, authz rules, service), tabel matriks hak akses beserta bukti curl, pengujian negatif, dan jawaban analisis C.4
Kode sumber	Diunggah ke repository masing-masing; tautan dicantumkan pada halaman pertama laporan
Batas waktu	Sebelum praktikum pertemuan berikutnya dimulai

Yang dinilai bukan banyaknya kode

Laporan yang memuat seluruh kode tetapi tanpa bukti pengujian akan dinilai lebih rendah daripada laporan dengan kode secukupnya namun disertai matriks hak akses yang lengkap dan pengujian negatif yang sungguh-sungguh dicoba.

Alasannya sederhana: kode authorization yang belum diuji adalah kode yang belum Anda ketahui perilakunya.

Catatan Penyusunan

Penyusunan modul ini dibantu oleh AI untuk pengembangan ide, penyusunan struktur materi, dan penulisan draf awal. Seluruh isi — termasuk kode program, skema database, status code, dan tabel matriks hak akses — telah dikurasi, dijalankan, dan diverifikasi ulang oleh dosen pengampu sebelum digunakan.

Seluruh potongan kode pada modul ini telah dikompilasi, diperiksa dengan go vet, dan dijalankan terhadap PostgreSQL 16 dengan tiga akun berperan berbeda. Setiap status code pada tabel matriks di Langkah 9 merupakan keluaran yang benar-benar diamati, bukan perkiraan.

Rujukan

- Dokumentasi Fiber — Middleware dan Routing: <https://docs.gofiber.io>
- OWASP Top 10 (2021), A01: Broken Access Control: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- OWASP Cheat Sheet Series — Authorization Cheat Sheet: <https://cheatsheetseries.owasp.org>
- NIST, Role Based Access Control (RBAC) — <https://csrc.nist.gov/projects/role-based-access-control>
- Dokumentasi PostgreSQL — Constraints dan Foreign Keys: <https://www.postgresql.org/docs/16/ddl-constraints.html>